APUNTES UNIVERSITARIOS

Foundations of Machine Learning and Deep Learning

Apuntes del Tema 9: Redes Neuronales Convolucionales (CNN I y II)

19 de octubre de 2026

Índice general

1 Apuntes del Tema 9: Redes Neuronales Convolucionales (CNN I y II) 2

1.1 1. De MLPs a Convoluciones: Motivación e Sesgos Inductivos 2

1.1.1 1.1 Limitaciones del Perceptrón Multicapa (MLP) en Imágenes 2

1.1.2 1.2 Principios de Invariancia Espacial 2

1.2 2. Operaciones Convolucionales y sus Parámetros 2

1.2.1 2.1 La Operación de Convolución 2D 2

1.2.2 2.2 Hiperparámetros Fundamentales: Padding, Stride y Canales 3

1.2.3 2.3 Ecuación de Dimensión de Salida 3

1.3 3. Capas de Agregación (*Pooling*) 3

1.3.1 3.1 Tipos Principales de Pooling 3

1.4 4. Jerarquía de Representaciones y Arquitecturas Clásicas 4

1.4.1 4.1 Extracción Jerárquica de Características 4

1.4.2 4.2 Arquitectura Clásica: LeNet-5 (Yann LeCun, 1998) 4

1.5 5. Gestión de Datos en PyTorch: `Dataset` y `DataLoader` 4

1.6 6. Práctica y Aplicación en Código (PyTorch y Torchvision) 5

1 Apuntes del Tema 9: Redes Neuronales Convolucionales (CNN I y II)

1.1 1. De MLPs a Convoluciones: Motivación e Sesgos Inductivos

1.1.1 1.1 Limitaciones del Perceptrón Multicapa (MLP) en Imágenes

- **Explosión de Parámetros:** Un MLP requiere aplanar la imagen de dimensión $C \times H \times W$ a un vector unidimensional $x \in \mathbb{R}^{C \cdot H \cdot W}$.
- *Ejemplo:* Para una imagen pequeña de $64 \times 64 \times 3 = 12\,288$ entradas, una primera capa oculta con $1\,024$ neuronas requiere $(12\,288 \times 1\,024) + 1\,024 \approx 12.58$ millones de parámetros.
- **Pérdida de Estructura Espacial:** Al aplanar la matriz de píxeles, el MLP trata la distancia entre píxeles adyacentes de la misma manera que la distancia entre píxeles situados en esquinas opuestas de la imagen.
- **Sensibilidad al Desplazamiento:** Un MLP no reconoce un objeto si se traslada unos pocos píxeles en la imagen, ya que la entrada vectorizada cambia por completo.

1.1.2 1.2 Principios de Invariancia Espacial

Para procesar señales estructuradas en 2D de forma eficiente, las Redes Neuronales Convolucionales (CNN) incorporan dos sesgos inductivos clave:

1. **Localidad (*Local Receptive Fields*):** Las características visuales significativas (bordes, esquinas, texturas) se forman por la interacción de píxeles contiguos en regiones espaciales reducidas.

1. **Invariancia a la Traslación / Compartición de Pesos (*Weight Sharing*):** La representación o patrón de un objeto no debe cambiar independientemente de la posición que ocupe dentro de la imagen. Un detector de bordes es útil tanto en la esquina superior izquierda como en el centro.

1.2 2. Operaciones Convolucionales y sus Parámetros

1.2.1 2.1 La Operación de Convolución 2D

Un filtro o núcleo (*kernel*) $K \in \mathbb{R}^{k_h \times k_w}$ se desliza sobre la matriz de entrada $X \in \mathbb{R}^{H \times W}$, calculando el producto escalar elemento a elemento seguido de una suma:

$$Y_{i,j} = (X * K)_{i,j} = \sum_m \sum_n X_{i+m,j+n} \cdot K_{m,n}$$

```
Entrada X (5x5)           Kernel K (3x3)          Feature Map Y (3x3)

[ 1  1  1  0  0 ]        [ 1  0  1 ]

[ 0  1  1  1  0 ]   *    [ 0  1  0 ]     --->     [ 4  3  4 ]

[ 0  0  1  1  1 ]        [ 1  0  1 ]              [ .  .  . ]

[ 0  0  1  1  0 ]                                 [ .  .  . ]
```

```
11
12 [ 0  1  1  0  0 ]
```

1.2.2 2.2 Hiperparámetros Fundamentales: Padding, Stride y Canales

- **Relleno (*Padding*, p):** Añade filas y columnas de ceros alrededor del borde de la entrada para controlar la dimensión espacial de salida y evitar la pérdida acelerada de información en los bordes.
- **Paso (*Stride*, s):** Especifica el número de píxeles que se desplaza el filtro en cada paso horizontal y vertical. Un $s > 1$ subsamplea la entrada reduciendo su dimensión espacial.
- **Múltiples Canales de Entrada y Salida ($C_{\text{in}}, C_{\text{out}}$):**
- Si la entrada tiene C_{in} canales (ej. $RGB = 3$), el filtro posee dimensiones $C_{\text{in}} \times k_h \times k_w$.
- Para extraer C_{out} mapas de características distintos, se aplican C_{out} filtros independientes de este tipo.

1.2.3 2.3 Ecuación de Dimensión de Salida

Dada una entrada de dimensión $C_{\text{in}} \times H \times W$, un kernel de tamaño $k_h \times k_w$, padding p y stride s, la dimensión del mapa de características resultante $C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}$ se calcula mediante:

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k_h}{s} \right\rfloor + 1$$

$$W_{\text{out}} = \left\lfloor \frac{W + 2p - k_w}{s} \right\rfloor + 1$$

1.3 3. Capas de Agregación (*Pooling*)

Las capas de pooling introducen **invariancia local a pequeñas traslaciones y rotaciones**, reduciendo la dimensión espacial de las representaciones y disminuyendo la carga computacional.

1.3.1 3.1 Tipos Principales de Pooling

1. **Max Pooling (`nn.MaxPool2d`):** Extrae el valor máximo dentro de la ventana de selección. Resalta las características más prominentes o activaciones más fuertes.
1. **Average Pooling (`nn.AvgPool2d`):** Calcula el promedio de todos los valores de la ventana. Proporciona una representación más suave de la región.

> **Propiedad importante:** Las capas de pooling operan de forma independiente sobre cada canal y **no poseen parámetros entrenables** (W, b).

1.4 4. Jerarquía de Representaciones y Arquitecturas Clásicas

1.4.1 4.1 Extracción Jerárquica de Características

A medida que la señal avanza a través de capas convolucionales profundas y subsampling:

- **Capas Iniciales:** Filtran patrones simples y locales de bajo nivel (bordes, gradientes de color, orientaciones).
- **Capas Intermedias:** Combinan los bordes para detectar texturas, esquinas y partes de objetos.
- **Capas Profundas:** Ensamblan las partes para representar objetos completos y estructuras semánticas complejas.

```

Imagen Raw    Bordes/Líneas    Texturas/Partes    Objetos Complejos    Clases (Logits)

 (Entrada)      (Capa Conv 1)     (Capa Conv 2)       (Capa Conv 3)     (Capas Densas
   )
```

1.4.2 4.2 Arquitectura Clásica: LeNet-5 (Yann LeCun, 1998)

Fue una de las primeras redes convolucionales con éxito comercial para la clasificación de dígitos manuscritos (MNIST).

- **Estructura Típica:**

1. **Entrada:** Imagen en escala de grises ($1 \times 28 \times 28$ o $1 \times 32 \times 32$).
1. **Bloque 1:** `Conv2d` ($1 \to 6$ canales, 5×5) + `Tanh` + `AvgPool2d` (2×2, $s = 2$).
1. **Bloque 2:** `Conv2d` ($6 \to 16$ canales, 5×5) + `Tanh` + `AvgPool2d` (2×2, $s = 2$).
1. **Clasificador denso:** Aplanado (`Flatten`) + `Linear` (120) + `Tanh` + `Linear` (84) + `Tanh` + `Linear` (10 clases).

1.5 5. Gestión de Datos en PyTorch: `Dataset` y `DataLoader`

Para alimentar eficientemente datos e imágenes a modelos de Deep Learning, PyTorch estandariza la carga mediante dos clases fundamentales:

1. `torch.utils.data.Dataset`**:** Almacena las muestras y sus etiquetas correspondientes. Debe implementar tres métodos:

- `__init__`: Inicializa rutas, transformaciones y metadatos.
- `__len__`: Devuelve el número total de muestras en el conjunto de datos.
- `__getitem__(self, idx)`: Carga y retorna una muestra (X, y) dado un índice idx.

1. `torch.utils.data.DataLoader`**:** Envoltorio que abstrae el procesamiento iterativo en minilotes (*mini-batches*), barajado de datos (*shuffling*), carga paralela multiproceso (*multiprocessing*) y gestión de memoria.

1.6 6. Práctica y Aplicación en Código (PyTorch y Torchvision)

A continuación se incluye una implementación modular completa que incluye la definición de un `Dataset` personalizado, la canalización de transformaciones, la arquitectura CNN tipo LeNet-5 y el bucle de entrenamiento:

```

import torch

import torch.nn as nn

import torch.optim as optim

from torch.utils.data import Dataset, DataLoader

import torchvision.transforms as transforms

from PIL import Image

import os



# ------------------------------------------------------------------------

# 1. Pipeline de Transformaciones y Preprocesamiento

# ------------------------------------------------------------------------

IMAGE_SIZE = 32



transform_train = transforms.Compose([

    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),

    transforms.RandomHorizontalFlip(p=0.5),

    transforms.ToTensor(),

    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

])



transform_val_test = transforms.Compose([

    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),

    transforms.ToTensor(),

    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

])


```

```
53
54 # ------------------------------------------------------------------------
55
56 # 2. Definición de la Arquitectura CNN (Inspirada en LeNet-5)
57
58 # ------------------------------------------------------------------------
59
60 class LeNet5Classifier(nn.Module):
61
62     def __init__(self, num_classes=10):
63
64         super(LeNet5Classifier, self).__init__()
65
66
67
68         # Bloque de Extracción de Características (Convolución + Pooling)
69
70         self.feature_extractor = nn.Sequential(
71
72             # Conv1: (B, 3, 32, 32) -> (B, 6, 28, 28)
73
74             nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5, stride=1, padding
    =0),
75
76             nn.Tanh(),
77
78             # Pool1: (B, 6, 28, 28) -> (B, 6, 14, 14)
79
80             nn.AvgPool2d(kernel_size=2, stride=2),
81
82
83
84             # Conv2: (B, 6, 14, 14) -> (B, 16, 10, 10)
85
86             nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5, stride=1,
    padding=0),
87
88             nn.Tanh(),
89
90             # Pool2: (B, 16, 10, 10) -> (B, 16, 5, 5)
91
92             nn.AvgPool2d(kernel_size=2, stride=2)
93
94         )
95
96
97
98         # Bloque Clasificador Denso
99
100        self.classifier = nn.Sequential(
101
102            nn.Flatten(), # (B, 16 * 5 * 5) = (B, 400)
103
104            nn.Linear(16 * 5 * 5, 120),
105
106            nn.Tanh(),
107
108            nn.Dropout(p=0.3),
```

```
            nn.Linear(120, 84),
            nn.Tanh(),
            nn.Linear(84, num_classes) # Logits de salida
        )

    def forward(self, x):
        x = self.feature_extractor(x)
        logits = self.classifier(x)
        return logits

# ----------------------------------------------------------------------
# 3. Bucle de Entrenamiento
# ----------------------------------------------------------------------
def train_cnn_demo():
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Modelo, Pérdida y Optimizador
    model = LeNet5Classifier(num_classes=10).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)

    # Tensor sintético de prueba: 16 imágenes RGB de 32x32
    dummy_images = torch.randn(16, 3, 32, 32).to(device)
    dummy_labels = torch.randint(0, 10, (16,)).to(device)

    model.train()
    optimizer.zero_grad()

    # Forward Pass
```

```
    logits = model(dummy_images)

    loss = criterion(logits, dummy_labels)



    # Backward Pass

    loss.backward()

    optimizer.step()



    print(f"Demostración exitosa en {device}.")

    print(f"Dimensión de Salida (Logits): {logits.shape}")

    print(f"Pérdida Inicial (Loss): {loss.item():.4f}")



if __name__ == "__main__":

    train_cnn_demo()
```

Dime si así te permite copiar todo el contenido sin problemas y si te gustaría que aclaremos algún concepto o que preparemos un cuestionario interactivo de este tema para preparar tu evaluación.